

Architectures web boostées à l'IA

Comment bien organiser, sécuriser et tester une appli qui utilise une IA

Comparaison Spring AI (Java) vs Laravel AI SDK (PHP)

Groupe : Quentin BAUDUIN, Timothé GEOFFROY, Kaan TOPKAYA, Erwin GOSSIN

Vue d'ensemble

Ce cours se déroule en 5 séances de 3 heures. Le but : apprendre à construire une appli web qui utilise une IA, sans que ça devienne un bazar impossible à maintenir.

On ne va pas traiter l'IA comme une fonctionnalité magique qu'on branche et qui marche toute seule. On va la traiter comme un morceau de code comme un autre : il faut l'organiser, le tester et le sécuriser au même titre que le reste de l'application.

Chaque séance mélange trois choses : un peu de théorie sur l'architecture, une comparaison entre deux façons de faire (Spring AI côté Java, Laravel AI SDK côté PHP), et un TP où vous faites évoluer le même projet de semaine en semaine, jusqu'à sa mise en ligne.

Dès la première séance, on installe aussi une IA qui tourne directement sur votre ordinateur, en plus de celle disponible sur internet. Comme ça, si vous voulez, vous pouvez faire tous les TP sans payer et sans clé d'accès. C'est aussi un bon exemple concret : le code doit pouvoir marcher avec l'IA en ligne ou l'IA locale sans qu'on ait à le réécrire on change juste "l'adaptateur" branché derrière.

Ce qu'on va savoir faire à la fin

À la fin du module, vous serez capables de :

- Considérer un agent IA comme un composant du code, et pas comme une boîte noire magique.
- Choisir une bonne façon d'organiser son code (architecture hexagonale, CQRS, événements) pour ajouter de l'IA sans tout coller ensemble.
- Comparer deux façons de brancher une IA sur une appli (Spring AI et Laravel AI SDK) avec des critères concrets : est-ce facile de changer de fournisseur, facile à surveiller, combien ça coûte.
- Tester une application dont les réponses ne sont jamais garanties identiques (l'IA ne répond pas toujours pareil, contrairement à du code classique).
- Repérer et limiter les risques : quelqu'un qui manipule l'IA avec un texte piégé, l'IA qui invente des choses fausses, des données confidentielles qui fuient, une facture qui explose, des réponses trop lentes.
- Justifier un choix technique (framework, outil, fournisseur d'IA) avec de vrais arguments d'ingénieur, pas juste parce que "c'est à la mode".

Ce qu'il faut déjà savoir

- Programmer en orienté objet (Java et/ou PHP), et connaître un minimum un framework web (Spring Boot et/ou Laravel).
- Savoir ce qu'est une API REST et comment marche une requête/réponse HTTP.
- Avoir une petite idée de ce qu'est une architecture logicielle (des couches, séparer les responsabilités).
- Savoir utiliser Git/GitHub au quotidien.

Le matériel nécessaire

- Un IDE, Git, et selon le parcours choisi : JDK 21+ / Maven ou Gradle (Spring), et/ou PHP 8.3+ / Composer (Laravel).
- Une clé d'accès chez un fournisseur d'IA en ligne (lequel sera précisé plus tard).
- Ou en alternative gratuite : une IA installée en local (par exemple avec Ollama), pour faire tous les TP sans payer. Machine recommandée : 16 Go de RAM, avec un petit modèle.
- Un endroit pour stocker des "embeddings" (des bouts de texte transformés en listes de nombres, pour pouvoir chercher par ressemblance) — par exemple pgvector — pour les TP sur le RAG.

Plan du cours

N°	Titre	Ce qu'on apprend
S1.1	Architectures web : les bases	Revoir les couches, la séparation des responsabilités, l'architecture hexagonale (isoler le cœur du code des outils externes) et le pattern CQRS (séparer lecture et écriture des données), appliqués à une appli web (contrôleur → service → domaine)
S1.2	Pourquoi l'IA complique les architectures classiques	Comprendre l'effet du fait qu'une IA ne répond jamais exactement pareil, met un temps variable à répondre, coûte à chaque appel, et dépend d'un service externe
S1.3	Installer son environnement : IA en ligne ou IA en local	Installer et configurer une IA en local (ex. Ollama) comme alternative à une IA en ligne ; brancher l'une ou l'autre au même endroit du code, et voir ce que ça change (prix, vitesse, confidentialité, TP faisables sans connexion)
S1.4	TP : réorganiser un code "fourre-tout"	Transformer un contrôleur qui fait tout vers une architecture plus propre ("ports/adapters"), parcours Spring ou Laravel, puis comparaison entre groupes
S2.1	Function / tool calling	Comprendre comment une IA peut "demander" à du code métier d'exécuter une action (nom, description, format des paramètres) ; comparer Spring AI et Laravel AI SDK
S2.2	Organiser le travail d'un agent	Mettre en place une "machine à états" pour un agent, un enchaînement d'appels, et un pattern pour annuler proprement en cas d'échec, pour une tâche en plusieurs étapes exposée par une API
S2.3	Le RAG comme pipeline de données	Concevoir : récupérer des documents, les découper, les transformer en vecteurs, les indexer, puis aller les rechercher pour les donner en contexte à l'IA ; comparer les outils de Spring AI et de Laravel
S2.4	TP : agent avec outils + recherche par similarité	Construire un agent (qui utilise des outils + une recherche par ressemblance) sur un cas simple, au choix avec Spring AI ou Laravel AI SDK
S3.1	Rendre son appli solide face à l'incertitude	Mettre en place : réessayer un appel qui échoue, un "coupe-circuit" qui bloque les appels si ça part en vrille, et un plan de repli (IA locale, puis mode sans IA)
S3.2	Surveiller ce que fait l'agent	Suivre chaque appel à l'IA, avoir des logs bien structurés, et des chiffres utiles (coût, vitesse, taux d'échec) dans une appli web
S3.3	TP : suivi + plan de secours	Ajouter le suivi et un mode de secours à l'agent construit en séance 2
S4.1	Tester un système qui ne répond jamais pareil	Simuler les réponses de l'IA, créer un jeu de tests de référence, utiliser une IA pour juger une autre IA, détecter quand la qualité baisse
S4.2	Risques et sécurité	Comprendre et limiter : les tentatives de manipuler l'IA avec un texte piégé (directement ou caché dans un document), les fuites de données, les inventions de l'IA dans un contexte sensible
S4.3	Comparer les approches	Comparer Spring AI, Laravel AI SDK et des outils dédiés (LangChain/LangGraph) sur : facilité à changer de fournisseur, facilité à surveiller, coût, maturité
S4.4	TP : écrire une série de tests pour l'agent	Écrire des tests (cas normal + tentatives de piégeage par texte) pour l'agent
S5.1	Gérer une IA en prod, différent du logiciel classique	Comprendre : versionner ses prompts, évaluer en continu, surveiller en prod — des choses qu'un cycle de dev classique ne prévoit pas
S5.2	Mise en prod et intégration continue adaptées	Adapter sa pipeline de tests automatiques à des résultats jamais parfaitement identiques, bien gérer les données ; comparer le déploiement en Java et en PHP
S5.3	Présentation finale	Présenter son architecture (10 min) en expliquant pourquoi ce framework, puis répondre aux questions

Détail des séances

Chaque séance suit la même trame : un peu de théorie, une comparaison Spring / Laravel, puis un TP. Les durées sont indicatives.

Séance 1 (3h) — Les bases de l'architecture web, et pourquoi l'IA les bouscule

S1.1 — Architectures web : les bases (45 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
10 min	Vidéo d'intro : où placer un appel à une IA dans une appli en couches, sans polluer le cœur du métier ?	Poser la question avant d'y répondre : laisser les étudiants proposer des idées	Regarde et se fait une première idée
20 min	Support écrit : les couches, la séparation des responsabilités, l'architecture hexagonale, le pattern CQRS, appliqués au flux contrôleur → service → domaine	Chaque notion est illustrée par un exemple en Java et un exemple en PHP	Lit le support et essaie les exemples fournis
15 min	Démo : le même besoin métier codé en Spring Boot puis en Laravel — repérer ce qui vient du framework et ce qui vient du métier	Le cœur du métier doit rester identique d'un framework à l'autre	Compare les deux versions et note les différences

S1.2 — Pourquoi l'IA complique les architectures classiques (45 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Le fait qu'une IA ne répond jamais exactement pareil, met un temps variable à répondre, coûte à chaque appel, et dépend d'un service externe	Relier chaque caractéristique à sa conséquence concrète sur l'architecture	Prend des notes organisées par caractéristique
15 min	Étude de cas : un appel à une IA fait en direct dans un contrôleur web — ce qui casse (timeout, worker bloqué, facture imprévisible)	Montrer un vrai cas de timeout, pas juste le décrire	Analyse le cas et repère les points de rupture
15 min	Discussion : quelles solutions possibles ? (traitement en arrière-plan, file d'attente, limite de temps, cache)	Ne pas conclure : les réponses seront construites en séance 3	Participe et teste ses idées

S1.3 — Installer son environnement : IA en ligne ou IA en local (40 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
10 min	Pourquoi une IA en local ? Pas de coût par appel, données qui restent chez soi, TP réutilisables sans connexion — contre une qualité et une vitesse différentes	Présenter le compromis honnêtement, sans dire que l'un est meilleur que l'autre	Écoute et identifie ce dont il a besoin
20 min	Atelier : installer une IA en local (ex. Ollama) et un petit modèle ; configurer en parallèle une clé d'accès à une IA en ligne	Prévoir une solution de secours pour les machines moins puissantes ; possibilité de bosser en binôme	Installe, télécharge un modèle et teste un premier appel
10 min	Démo clé : brancher l'IA locale et celle en ligne au même endroit du code — deux façons différentes, un seul cœur métier	C'est le moment où l'idée de "ports & adapters" devient concrète	Change de fournisseur d'IA en changeant une seule ligne de configuration

S1.4 — TP : réorganiser le code (50 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
10 min	Présentation du code de départ : un contrôleur "fourre-tout" qui appelle directement le SDK du fournisseur d'IA	Le code fourni doit être réaliste, pas exagéré	Lit le code et repère les endroits trop liés entre eux
30 min	Réorganisation guidée : sortir le cœur métier, écrire un adaptateur pour le fournisseur d'IA, isoler le métier	Chaque groupe travaille sur Spring ou Laravel selon son parcours	Réorganise le code et fait passer les tests fournis

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
10 min	Mise en commun : ce que le framework impose, ce qu'il laisse libre	Faire ressortir les principes qui restent vrais peu importe le langage	Présente sa solution et écoute l'autre parcours

Séance 2 (3h) — Un agent IA, ça se construit comme un composant

S2.1 — Function / tool calling (45 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Le tool calling : un contrat entre l'IA et le code — nom, description, format des paramètres ; l'IA propose, l'appli exécute	Insister : l'IA n'exécute rien elle-même, elle demande	Prend des notes sur le déroulement d'un appel
20 min	Comparaison : déclarer un outil avec Spring AI (ChatClient, annotations) et avec Laravel AI SDK (classe d'agent, méthode tools())	Mettre les deux exemples côte à côte sur le même outil	Lit, teste, et repère ce qui se ressemble
10 min	Point sécurité : un outil, c'est une porte ouverte vers le code métier — donner le minimum de droits nécessaire	Annonce la séance 4 sans trop en dire	Liste les outils qu'il n'exposerait pas

S2.2 — Organiser le travail d'un agent (40 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	La boucle d'un agent, la machine à états, l'enchaînement d'appels et un pattern pour annuler proprement en cas d'échec, pour des tâches en plusieurs étapes	Distinguer ce qui est décidé à l'avance de ce qui est décidé par l'IA	Dessine la machine à états d'un exemple simple
15 min	Étude de cas : une tâche en trois étapes où une seule a vraiment besoin de l'IA	Montrer qu'on limite ce qu'on confie à l'IA	Repère où l'IA est utile, et où elle ne l'est pas
10 min	Exposer ça via une API web : requête longue, streaming, ou file d'attente	Relier au problème de lenteur vu en S1.2	Choisit une méthode et explique pourquoi

S2.3 — Le RAG comme pipeline de données (45 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Le RAG, ce n'est pas une fonctionnalité magique de l'IA, mais un pipeline de données : récupérer, découper, transformer en vecteurs, indexer, rechercher, donner en contexte	Traiter ça avec la même rigueur qu'un pipeline de données classique	Dessine le pipeline en entier
20 min	Comparaison : les outils VectorStore de Spring AI vs la recherche par similarité (pgvector) du Laravel AI SDK	Montrer que la base vectorielle n'est qu'un adaptateur de plus	Indexe un petit ensemble de documents et fait une recherche
10 min	Les limites : le RAG n'empêche ni les inventions de l'IA, ni le risque qu'un document piégé la manipule	Semer le doute avant la séance 4	Trouve une question à laquelle son index répond mal

S2.4 — TP : agent avec outils + recherche (50 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
10 min	Présentation du cas fil rouge, réutilisé jusqu'à la séance 5	Cas simple, compréhensible sans expertise préalable	Découvre le cas et les critères de réussite

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
35 min	Construire un agent avec un ou deux outils, plus une recherche par similarité sur un ensemble de documents fourni	Le cœur métier ne doit dépendre d'aucun fournisseur d'IA en particulier ; l'IA locale doit rester utilisable	Code, teste à la main, versionne son travail
5 min	Point d'étape : ce qui marche, ce qui échoue de façon imprévisible	Noter les échecs : ils serviront en S3 et S4	Note les comportements bizarres observés

Séance 3 (3h) — Rendre son appli fiable et surveillable

S3.1 — Rendre son appli solide face à l'incertitude (55 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Réessayer un appel qui échoue (avec délai qui augmente), coupe-circuit, limite de requêtes en parallèle — et la question du plan de secours	Réessayer un appel payant coûte deux fois : le rappeler	Prend des notes et estime le coût d'une politique de réessai
20 min	Comparaison : Resilience4j côté Spring vs les mécanismes natifs de Laravel (bascule automatique entre fournisseurs d'IA, files d'attente)	Le SDK Laravel gère nativement le changement de fournisseur : montrer ce que ça évite d'écrire	Compare les deux façons de faire sur la même panne simulée
20 min	Construire un plan de repli à trois niveaux : IA en ligne → IA locale → mode sans IA	Le mode sans IA doit rendre un vrai service, pas juste afficher une erreur	Définit ce que fait son appli quand toute IA est indisponible

S3.2 — Surveiller ce que fait l'agent (50 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Le suivi ("tracing") appliqué à une chaîne d'appels IA (un suivi par appel, par outil, par recherche), logs bien structurés, tout relié par un identifiant de requête	Un suivi doit permettre de comprendre après coup pourquoi l'agent a fait ce choix	Repère ce qu'il faut suivre sur son propre agent
20 min	Les chiffres qui comptent : vitesse par étape, nombre de tokens utilisés, coût par requête, taux d'échec, taux de recours au plan de secours	Suivi natif chez Spring AI vs instrumentation via les événements du SDK Laravel	Choisit ses indicateurs et explique pourquoi
15 min	Le coût comme contrainte d'architecture : budget par requête, plafond, alerte si ça dérape	Un coût qu'on ne mesure pas est un coût qu'on ne maîtrise pas	Estime le coût mensuel de son agent pour un usage donné

S3.3 — TP : suivi + plan de secours (55 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
40 min	Ajouter à l'agent de la séance 2 : suivi complet de la chaîne, logs structurés, chiffres de coût/vitesse, puis le plan de secours à trois niveaux	Vérifier le plan de secours en coupant vraiment le réseau vers l'IA en ligne	Instrumente, provoque la panne, observe ce qui se passe
15 min	Analyse des traces produites : où passe le temps, où part l'argent	Comparer ses intuitions de la séance 1 aux vraies mesures	Commente ses propres résultats et trouve une piste d'amélioration

Séance 4 (3h) — Bien tester, et se méfier des attaques

S4.1 — Tester un système qui ne répond jamais pareil (55 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Pourquoi vérifier une égalité exacte ne suffit plus ; revoir la pyramide de tests pour un composant imprévisible	Séparer ce qui est prévisible (le code) de ce qui ne l'est pas (la réponse de l'IA)	Classe les parties de son agent en prévisibles ou non
20 min	Techniques : simuler les réponses de l'IA, créer un jeu de tests de référence, utiliser une IA comme "juge", tester si le sens des réponses dérive, définir une marge de tolérance	Une IA-juge n'est elle-même pas fiable à 100 % : ne pas s'y fier aveuglément	Crée dix cas de test de référence sur son propre sujet
20 min	Comparaison d'outils : JUnit + Evaluator API de Spring AI vs PHPUnit/Pest + les simulateurs du SDK Laravel (Agent::fake())	Le simulateur "prêt à l'emploi" de Laravel rend le test simple : le montrer	Écrit un premier test avec un simulateur

S4.2 — Risques & sécurité (50 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Le référentiel OWASP Top 10 pour les applis IA, avec l'injection de prompt (manipulation par texte piégé) en première place depuis deux éditions	La cause : les instructions et les données passent par le même canal	Situe son agent face à chacun des dix risques
20 min	Injection directe (l'utilisateur écrit l'attaque lui-même) et indirecte (un document indexé la contient) ; fuite de données via le contexte ; outils avec trop de droits	Montrer une vraie injection indirecte sur l'index construit en séance 2	Écrit un texte piégé et le teste sur son propre agent
15 min	Se défendre sur plusieurs niveaux à la fois : vérifier les entrées, filtrer les sorties, donner le minimum de droits aux outils, séparer le contenu non fiable, faire valider par un humain les actions sensibles	Aucune mesure seule ne suffit : c'est le cumul qui compte	Choisit deux mesures et les met en place

S4.3 — Comparer les approches (30 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Grille de comparaison : Spring AI, Laravel AI SDK, LangChain/LangGraph — facilité à changer de fournisseur, suivi natif, facilité à tester, maturité, écosystème	Comparer sur des critères techniques, jamais sur la mode	Remplit la grille pour son propre projet
15 min	Cas de décision : quand une seule stack suffit, et quand un service Python séparé se justifie vraiment	Un service séparé a un vrai coût de fonctionnement : le nommer	Défend un choix et répond aux objections

S4.4 — TP : écrire une série de tests (45 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
30 min	Écrire une série de tests : cas normal avec réponses simulées, cas dégradé, et au moins deux tentatives d'injection (directe et indirecte)	Un test d'injection qui passe du premier coup est probablement mal écrit	Écrit les tests, les fait échouer, puis corrige l'agent
15 min	Intégrer au dépôt et lancer toute la suite	Mesurer le temps d'exécution : ça conditionne la CI de la séance 5	Commit, exécute, note la durée

Séance 5 (3h) — Mettre en prod et présenter son travail

S5.1 — Gérer une IA en prod (45 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
15 min	Ce que le cycle classique ne couvre pas : le prompt est aussi un fichier à versionner, l'évaluation est continue, la qualité peut baisser sans qu'on touche au code	Une mise à jour du modèle côté fournisseur est un déploiement qu'on ne contrôle pas	Repère ce qu'il faut versionner dans son projet
15 min	Versionner ses prompts, gérer les versions de modèle, figer une version, prévoir une migration en cas de changement	Figer la version du modèle n'est pas optionnel en prod	Met en place un versionnage de ses prompts
15 min	Surveillance en prod : baisse de qualité, dérive du coût, retours des utilisateurs	Distinguer un incident ponctuel d'une dérive lente	Définit trois alertes et leurs seuils

S5.2 — Mise en prod & CI/CD adaptées (45 min)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
20 min	Adapter sa pipeline : quels tests bloquent la mise en prod, lesquels tournent en arrière-plan ? Gérer le coût et la lenteur des tests qui appellent une vraie IA	Faire tourner l'évaluation coûteuse à côté du chemin principal, mais la faire quand même tourner	Conçoit sa pipeline en deux étapes
15 min	L'IA locale comme outil de CI : évaluer sans clé d'accès ni facture, au prix d'une fidélité un peu moindre	Ça rejoint la séance 1 : l'architecture par adaptateurs paie ici	Évalue si cette option est utile pour son projet
10 min	Gestion des données, traçabilité des prompts et réponses, contraintes légales ; déploiement comparé Java vs PHP	Ce qui part dans le contexte de l'IA quitte l'entreprise : le rappeler une dernière fois	Note quelles données passent par son agent

S5.3 — Présentation finale (1h30)

Durée	Contenu	Point d'attention	Ce que fait l'étudiant
60 min	Présentation par groupe (10 min) : architecture choisie, pourquoi ce framework, stratégie de test, mesures de sécurité, coût estimé	Chaque groupe doit défendre un choix, pas juste décrire ce qu'il a codé	Présente et répond aux questions
20 min	Questions croisées entre groupes Spring et Laravel sur les mêmes problèmes	C'est là que la comparaison prend tout son sens	Interroge un groupe de l'autre parcours
10 min	Bilan : ce qui vient de l'architecture, ce qui vient du framework, ce qui vient du modèle d'IA	Conclure sur l'idée principale : un agent IA est un composant comme un autre	Dit ce qu'il retient et ce qu'il ferait différemment

Pour aller plus loin (bibliographie)

Références utilisées pour construire ce module. Celles avec une (★) sont à lire en priorité.

Architecture logicielle — les bases

Alistair Cockburn, « Hexagonal Architecture »

l'article d'origine sur l'architecture "ports & adapters" (isoler le cœur du code des outils externes), court et toujours utile.

Martin Fowler, « CQRS »

explication simple du pattern qui sépare la lecture et l'écriture des données, avec des cas concrets.

Michael Nygard, Release It!

le livre qui a popularisé le coupe-circuit, le bulkhead et les timeouts : les bases de la résilience.

Frameworks et SDK (★)

Spring AI — Documentation officielle (★)

ChatClient API, Advisors API, VectorStore, pipeline d'ingestion, suivi, Evaluator API. Support natif d'Ollama pour tourner en local.

Laravel AI SDK — Documentation officielle (★)

le package officiel laravel/ai, sorti en février 2026. Création d'agents (make:agent), tool calling, sorties structurées, streaming, embeddings et recherche par similarité (pgvector), simulateurs de test, bascule automatique entre fournisseurs.

Resilience4j — Documentation

coupe-circuit, réessai, limite de débit et bulkhead pour l'écosystème Java (séquence S3.1).

Faire tourner une IA en local

Ollama — Documentation (★)

l'outil pour faire tourner une IA en local, utilisé en séquence S1.3. Supporté par Spring AI et par le SDK Laravel.

pgvector

extension PostgreSQL pour la recherche vectorielle, utilisée dans les TP de RAG des deux parcours.

Sécurité et risques (★)

OWASP Top 10 for LLM Applications, édition 2025 (★)

le référentiel central de la séquence S4.2. L'injection de prompt est en première place pour la deuxième fois de suite : une IA reçoit les instructions et les données par le même canal, et ne peut techniquement pas faire la différence entre les deux. Le document rappelle qu'aucune solution ne règle ce problème définitivement, et conseille de se défendre sur plusieurs niveaux à la fois. L'édition 2025 ajoute une catégorie sur les pipelines RAG, où quelqu'un peut empoisonner la base de documents.

OWASP Top 10 for Agentic Applications

la suite du précédent pour les systèmes à base d'agents : détournement d'objectif, abus des outils, empoisonnement de la mémoire.

Greshake et al., « Not what you've signed up for »

la démonstration d'origine de l'injection indirecte via des documents récupérés — la base de l'exercice pratique de S4.2.

Suivi et gestion en prod (LLMOps)

OpenTelemetry — Semantic Conventions for GenAI

des conventions de nommage pour suivre les appels IA.